

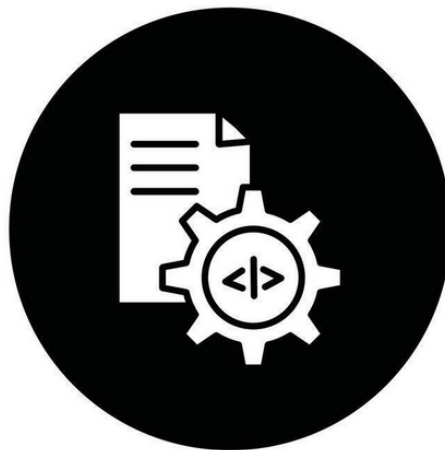
ENSSAT Lannion

Année Universitaire 2025-2026

INFO2

Théorie des langages et Compilation

Compte-rendu projet réalisation d'un compilateur



Travail réalisé par :

Florian ABADIE

Chloé AUBRY

Quentin BRULÉ

Clem JOURDIN

Table des matières

Introduction	3
Schéma de compilation	4
Étapes du projet	5
Construction de la table des identificateurs	5
Conception et écriture de la machine virtuelle	5
Difficultés rencontrées	5
Vérifications sémantiques	5
Difficultés rencontrées	6
Génération du code objet	6
Architecture générale	6
Stockage des instructions générées	6
Points de génération	6
Difficultés rencontrées et choix techniques	7
Conclusion et perspectives	7
Bibliographie	7

Introduction

L'objectif de ce projet est de réaliser un compilateur pour le langage NilNovi Procédural.

Tout d'abord, l'analyseur lexical est chargé d'interpréter le code source du programme et de le segmenter en unités lexicales. Les différentes unités lexicales sont alors reconnues si elles correspondent aux types prévus. La suite de ces éléments reconnus par l'analyseur lexical constitue la chaîne codée qui est fournie en entrée de l'analyseur syntaxico-sémantique. Ce dernier est par la suite chargé de réaliser l'analyse syntaxique et l'analyse sémantique. En parallèle de l'analyse syntaxique, la table des identificateurs est complétée au fur et à mesure. Elle permet de stocker les différentes informations nécessaires aux vérifications sémantiques et à la génération de code. Enfin, l'analyseur syntaxique pilote la génération de code qui s'effectue en parallèle.

Schéma de compilation

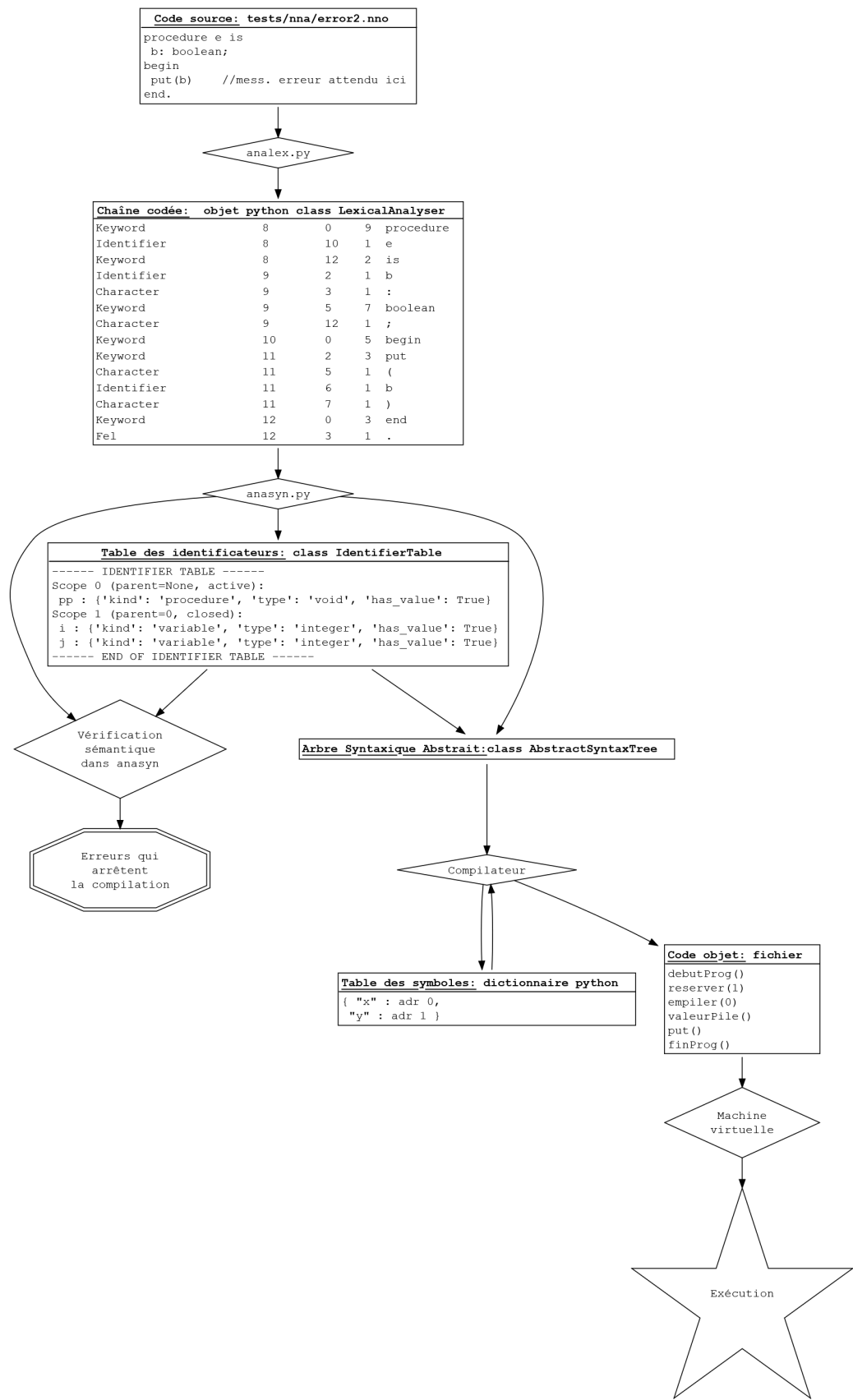


Fig. 1. – Schéma de compilation de notre projet

Étapes du projet

Construction de la table des identificateurs

La table des identificateurs est un arbre de portées, dans lequel chaque portée correspond à un bloc du programme. Pour chaque identificateur, la table indique sa nature (variable, procédure, fonction...), son type (entier ou booléen), un booléen indiquant si l'identificateur est initialisé ou non, ainsi que, pour les paramètres, leur mode (in ou in out). La portée 0 est la portée globale.

Lorsqu'un bloc est analysé, une nouvelle portée enfant est créée, dans laquelle sont stockées par exemple les variables locales comme *i* et *j*. Une fois le bloc terminé, la portée est marquée comme « closed », ce qui signifie qu'elle n'est plus active mais reste visible dans l'arbre pour inspection. Cette représentation permet de visualiser clairement la hiérarchie des portées, les déclarations associées et la manière dont les identificateurs sont organisés pendant l'analyse.

Conception et écriture de la machine virtuelle

Pour réaliser la machine virtuelle, la ressource la plus importante est l'annexe PDF au projet, intitulée « nilNovi-15-16.pdf ».

Lors de la lecture de cette annexe, il est apparu que la gestion fine de la mémoire deviendrait importante, et comme le choix du langage d'implémentation de la machine virtuelle est libre, le langage C est paru comme un langage tout à fait adapté pour cette tâche.

Une première machine virtuelle a été réalisée pour le langage NilNovi Algorithmique, puis une seconde pour le langage NilNovi Procédural, car la deuxième machine est essentiellement une copie de la première avec quelques fonctions en plus.

Difficultés rencontrées

Une difficulté rencontrée est l'inexactitude de l'annexe : les exemples de code objet pour la partie NilNovi Algorithmique et Procédural sont faux et l'exécution est incohérente. S'en rendre compte est compliqué et j'ai dû effectuer des compilations à la main pour m'en rendre compte.

Vérifications sémantiques

NilNovi Procédural est un langage fortement typé reconnaissant 2 types : les entiers et les booléens. Le typage fort implique qu'une variable ne peut pas changer de type. Tout identificateur doit être déclaré avec son type avant de pouvoir être utilisé dans sa portée.

L'objectif principal de cette étape est de s'assurer que ces propriétés de NilNovi Procédural sont bien respectées.

Ces vérifications s'effectuent au sein de `anasyn.py`, en parallèle de la construction de la table des identificateurs. On se sert également de la table des identificateurs pour connaître les types et valeurs des variables du programme au sein de leur portée et ainsi s'assurer que les propriétés du langage mentionnées plus haut sont bien respectées.

De manière plus globale, cette étape permet de vérifier si les règles de la grammaire du langage sont bien respectées. La vérification sémantique se base donc sur les règles de la grammaire de NilNovi Procédural, présentes dans la partie annexe du sujet du projet : « `Projet_THL_Compilation.pdf` ».

Difficultés rencontrées

La principale difficulté de cette étape est de s'assurer de l'exhaustivité des tests de vérification sémantique. L'utilisation des règles de grammaires du programme a permis de s'assurer de cette exhaustivité en vérifiant, une à une, que chaque règle était respectée.

Génération du code objet

La génération de code constitue la dernière phase du compilateur : elle transforme la représentation intermédiaire. Ici, l'arbre syntaxique abstrait (AST) en une séquence d'instructions directement exécutables par la machine virtuelle NilNovi (MV). Cette section décrit les choix de conception, les mécanismes mis en oeuvre et les difficultés rencontrées lors de l'implémentation de cette phase en Python.

Architecture générale

Nous avons choisi de mettre en place une architecture en couches pour le compilateur ainsi la génération de code est entièrement séparée de l'analyse syntaxique. L'analyseur syntaxique (`anasyn.py`) construit l'AST et remplit la table des identificateurs (`IdentifierTable`), puis, une fois l'arbre complet, le générateur de code (`GenerateurCodeNilNovi.py`) prend possession de l'arbre et le parcourt.

Ce parcours repose sur le **patron visiteur** (*visitor pattern*). Chaque noeud de l'AST expose une méthode `accept(visiteur)`, qui effectue une résolution dynamique du nom de la méthode à invoquer :

```
def accept(self, visiteur):
    nom = f"visit_{type(self).__name__}"
    return getattr(visiteur, nom)(self)
```

Le générateur définit une méthode `visit_<NomDuNoeud>` pour chaque type de noeud du langage (`visit_Programme`, `visit_Si`, `visit_TantQue`, `visit_Affectation`, etc.). Cette architecture facilite le travail lorsque l'on souhaite ajouter de nouvelles méthodes : ajouter un nouveau noeud ne requiert que l'ajout d'une méthode supplémentaire dans le générateur, sans modifier l'arbre.

Stockage des instructions générées

Toutes les instructions produites sont accumulées dans une liste Python `self.instructions` (de type `list[str]`), l'indice d'un élément correspondant directement à son adresse dans le programme cible. La méthode centrale `emettre(instruction)` encapsule l'ajout en fin de liste :

```
def emettre(self, instruction: str):
    self.instructions.append(instruction)
```

Une fois l'arbre intégralement parcouru, la méthode `sauvegarder(chemin)` écrit chaque instruction sur une ligne distincte du fichier de sortie, produisant ainsi le programme objet lisible par la machine virtuelle.

Points de génération

Un point de génération est un noeud de l'AST dont le traitement implique l'émission d'une ou plusieurs instructions MV. L'identification de ces points suit la sémantique opérationnelle de chaque construction du langage NNP, formalisée lors de la conception de l'AST.

Les principaux points de génération sont les suivants :

- **Début et fin de programme** (`visit_Programme`) : encadrement par `debutProg()` et `finProg()`.
- **Déclarations de variables** (`visit_DeclarationVariables`) : réservation de l'espace mémoire via `reserver(n)`.
- **Expressions arithmétiques et booléennes** (`visit_Nombre`, `visit_Identifiant`, `visit_OperationBinaire`, etc.) : empilement de valeurs et application d'opérateurs.
- **Affectation** (`visit_Affectation`) : calcul de l'adresse cible, évaluation de l'expression source, instruction `affectation()`.
- **Structures de contrôle** (`visit_Si`, `visit_TantQue`) : sauts conditionnels (`tze`) et inconditionnels (`tra`).
- **Appels de procédures et de fonctions** (`visit_AppelProcEDURE`, `visit_AppelFonction`) : réservation du bloc d'activation et branchement.
- **Instruction retourner** (`visit_Retourner`) : émission de `retourFonct()`.
- **Entrées/sorties** (`visit_Ecrire`, `visit_Lire`) : instructions `put()` et `get()`.

Difficultés rencontrées et choix techniques

La principale difficulté a été de déterminer quelles informations récupérer, à quel moment, et depuis quelle source. La table des identificateurs construite par l'analyseur syntaxique (`IdentifierTable`) est orientée vers la vérification sémantique (types, portées, modes de passage) et n'expose pas directement les adresses de pile dont la génération a besoin.

La solution retenue a été de doter le générateur de sa propre table des symboles, construite en parallèle lors du parcours de l'AST. Cette redondance est en réalité un choix délibéré : les deux tables servent des objectifs distincts. La table du générateur associe chaque identifiant à une adresse concrète sur la pile, information qui n'existe pas avant la phase de génération.

Conclusion et perspectives

Ce projet nous a permis de détailler le processus de compilation à travers 4 étapes distinctes, chaque étape est interconnectée afin de produire le code objet exécutable par la machine virtuelle. Néanmoins, le langage Python était imposé pour la création du compilateur, il serait intéressant d'utiliser un langage plus bas niveau comme le langage C afin de réduire le temps d'exécution et de comparer les résultats face à la solution proposée précédemment.

Bibliographie

<https://pablo.rauzy.name/teaching/ic/05-analyse-semantique.pdf>
https://henri-garreta.developpez.com/tutoriels/techniques-outils-compilation/?page=page_4
https://fr.wikipedia.org/wiki/Arbre_de_la_syntaxe_abstraite
https://foad.univ-rennes.fr/pluginfile.php/1223260/mod_resource/content/1/nilNovi-15-16.pdf
https://en.wikipedia.org/wiki/Visitor_pattern
https://en.wikipedia.org/wiki/Abstract_syntax_tree